Task 6 — Long-Horizon Memory and Context Engineering

TRACK Agentic AI — Intern Programme, Task 6 of 8

PACKAGE	packages/memory
PREREQUISITES	Task 3 (agent loop) and Task 4 (orchestrator). TypeScript.
COST	Zero. Everything runs locally.

Why this task exists

Every agent built so far ran inside one bounded session with a step budget of a dozen or so. Real work doesn't fit in one context window — a task that spans days, a codebase too large to ever load in full, a decision made 200 turns ago that still matters now. The naive fix is “make the context window bigger,” which is a real trend (models now take well over a million tokens) and also a trap: a bigger window doesn't mean the model uses the middle of it well — you already met “lost in the middle” in Task 2 — and it doesn't mean you should pay to re-read everything on every step.

Memory is not “keep everything.” Memory is deciding, explicitly, what a future step needs and what it can safely forget. That decision, made badly, is invisible until the agent contradicts something it already decided, or forgets a constraint that mattered.

Setup

Same runtime as prior tasks: Node 20+, TypeScript strict, Vitest, Ollama.

What you are building

Extend the Task 3 agent to run against a task that cannot be solved within a single context window — e.g. a multi-file refactor across 40+ files, or a task explicitly split across multiple sessions with the process restarted between them (“close the laptop, come back tomorrow,” simulated by literally killing the process and reloading state from disk).

```
pnpm memory run --task "rename the auth module and update all 40+ call sites"
pnpm memory eval
pnpm memory eval --compare baseline.json
```

Requirements

1. Tiered memory

Working memory: the current step's context, bounded, always fits. Episodic memory: a log of what happened this run, summarized — not verbatim — once it exceeds a size threshold. Persistent memory: facts worth keeping across runs entirely, written explicitly by the agent, not produced by dumping the whole transcript.

2. Compaction, not truncation

When working memory would overflow, summarize the oldest portion instead of silently dropping it. Write down, in NOTES.md, what a summarization pass predictably loses — and test for it, don't just assert it.

3. Retrieval into context

Persistent memory is retrieved — reuse your Task 2 hybrid retrieval — not injected wholesale. The agent asks for what it needs for the current step, the same way it would query any other index.

4. Cold-restart correctness

Kill the process mid-run and restart it from persisted state. The agent must resume without repeating completed work or contradicting a decision it already made — both are the actual failure modes memory is supposed to prevent.

5. Golden evaluation set — 12 long-horizon scenarios

CATEGORY	COUNT	DEFINITION
Standard long-horizon	7	Cannot complete in one working-memory window; requires compaction to finish
Recall-dependent	3	Correct completion requires recalling a specific decision made much earlier in the run
Memory pollution	2	A lot of irrelevant noise is fed early on; correct behavior is to not resurface stale, now-wrong information from it

6. Metrics

METRIC	WHAT IT MEASURES
resume correctness	No repeated or contradicted work after a cold restart
context-budget adherence	Working memory never exceeds its cap
recall accuracy	Correctness on the “remember this decision” cases
memory pollution rate	Stale or irrelevant facts incorrectly resurfacing

Deliverable: RESULTS.md

A table comparing: no memory management (fails past N steps) vs naive truncation vs summarization only vs tiered memory with retrieval. Attribute: if summarization alone fixes the budget problem but recall accuracy stays low until retrieval is added, say so and explain why the two aren't redundant.

Hard parts you should expect

- Summarization that quietly drops the one constraint that mattered — the summary reads fine and is still wrong in a way you won't notice without a targeted eval case for it.

- Deciding what's “worth persisting” without just persisting everything — that's not memory, that's a bigger window with extra steps.
- An agent that trusts a stale persisted fact over what it can currently see is true — persisted memory needs an implicit or explicit staleness signal.
- Cold-restart bugs that only show up when the kill happens mid-tool-call, not between steps — test the ugly timing, not just the clean one.

Deliverables

- packages/memory — tiered memory store, compaction, retrieval integration
- evals/golden-memory.jsonl — 12 long-horizon scenarios
- Metrics harness reporting all four metrics
- RESULTS.md, NOTES.md (what summarization loses and how you mitigated it), DESIGN.md

Design doc, before any code

One page, reviewed and approved before you write a line:

- Public interfaces and types (memory tiers, compaction trigger, retrieval query shape)
- The three most likely failure modes, and your plan for each
- What you are deliberately not building, and why
- Open questions

Review gate

Your reviewer will verify these by hand before merge.

- Kill a run mid-tool-call and restart it; confirm no step is silently repeated or contradicted.
- Feed a memory-pollution case and confirm the stale fact does not resurface in the final answer.
- Force working memory past its cap and confirm compaction fires before any hard failure.
- Every number in RESULTS.md reproduces from a fresh clone.